微软 Copilot 团队的企业级 AI 开发实践

分享人：李荣老师（微软 Copilot 团队） | 录音时间：2026/10/22 20:41

团队背景

- 李荣老师所在团队负责 Copilot 客户端的开发，主要面向非微软开发环境，包括 JetBrains 全家桶、Eclipse、Xcode 等。
- 团队规模为几十人，代码仓库规模为国外水平，客户群规模为几百万。
- 产品为付费商业软件，用户包括个人用户和相当比例的企业用户，对质量要求非常高，且服务对象为同行，对软件质量要求更高、更挑剔。

AI 应用初期体验

- 团队在去年年初上半年推出 agent 功能，开发者可将在 GitHub 上的 issue 直接 assign 给 Copilot，其自动工作。
- 使用 agent 生成代码后，开发者查看运行结果并测试，头 3 个月未发现问题，工作效率大幅提升，一天工作约 2 小时即可完成，管理层对此满意。

问题爆发与应对

- 两三个月后，客户抱怨越来越多且明显，包括应用市场差评、售后团队转来的问题升级（escalation）增多，企业用户以退订施压。
- 团队在去年接近年底时进行大刹车，几乎停止所有新功能开发，专门聚焦质量及内部架构调整，付出非常大的代价。
- 问题的根源在于有人的地方就存在信息差，每个人指挥 agent 时，信息差会被放入 code base，agent 会不断重复这些差异，加上内卷环境，AI 使所有人卷起来。

工作职责转变

- AI 能生成大量代码，但人的注意力有限，传统评审环节在 AI 快速生成代码的速度下，人无法逐行阅读，出现三种情况：逐行仔细查看、完全不看、看似在看但实际看得很浅。
- 开发者工作重心发生重大变化，写代码的工作基本由 AI 完成，人的工作转向**澄清问题（输出 spec）、技术设计（design）、验证过程（shift left，验证越早发生越好）和最终签字（sign off）负责**。

质量保障与架构控制

- **守住质量底线**：AI 可根据 spec 生成对应 test case，根据 call chain 预测需要运行的 test case，在全集 test case 的子集上做快速验证。
- **控制架构偏移**：资深工程师需盯着每个 PR 对架构的偏移影响，防止重复代码过多；两种解决思路为更好的阶段控制和利用 AI 快速生成代码的能力生成新系统，后者在系统边界清晰时是更高效的重构方法。
- **释放速度**：在质量和技术控制后，可考虑将事情推向更高速方向，全部交给 AI，但需有足够保障，涉及 harness engineer 和 loop engineer 概念。

团队与个人循环

- **团队循环**：PM 从客户需求和商业价值角度提供 feature 需求，测试发现 bug，客户告知 bug，遥测提供信号，AI 自动监控指标并以 issue 形式放入中心化 backlog，AI 可处理问题分诊、档案和优先级，但最终由人工判断。
- **个人循环**：组员通过 iteration planning 从 backlog 中拿问题，处理流程为理解形成 spec、形成设计、让 agent 生成代码、完成验证。
- 最终审核关注**证据差异**而非代码本身，通过交叉验证各环节证据是否吻合，差异判断由人完成。

团队记忆与知识管理

- backlog 是事实上的团队 memory，GitHub 的 issues、wiki、PR 和 code 构成整个 memory。
- 设计原则、编码规则等确定性内容以 instruction 形式存在代码仓库，动态功能内容以 issue 形式存在。
- 必须保持 backlog 清洁，及时丢弃无价值信号，六个月以上的 issue 若无客户强制要求可关闭。

组织与人员影响

- 有相当经验的人使用 agent 后效率极高，如 Meta 朋友处听到人日均 PR 水平为 30，社交媒体上还有 200、900 等数字。
- 刚入行的人面临学习路径被抹掉的问题，无法通过踩坑积累判断力，最大障碍是不知道自己的无知，无法提出正确问题。
- 组织影响包括倡导组织扁平化、拆掉海尔体系，有些公司已不再招 junior developer，新人培养是开放式问题。

现场问答

- **关于个人绩效到组织绩效转化**：李荣老师认为关键在于团队共享 memory 的维护，如会议记录、文档等信息应自动暴露给 agent，微软有 work IQ 产品可查询 M365 文档和会议记录。

- **关于代码质量保证**：团队有 daily report 汇总当天所有 PR 的架构变化，通过持续迭代调整 agent 的 instruction 来维护生成代码的机器。
- **关于 DRI 是否会成为瓶颈**：李荣老师强调 shift left 概念，设计领域问题由专门 agent 识别并触发相应 DRI 参与讨论，DRI 在特定领域有经验，同一领域由同一人做决定可降低自相矛盾概率。